

Flashing and Debugging Your Nucleo

Installing the Necessary Tools

In order to flash and debug your Nucleo with west, you need to install the proper tooling first. You will need to install the following:

Tool	Description	Notes
STM32CubeProgrammer	STM32CubeProgrammer (STM32CubeProg) is an all-in-one multi-OS software tool for programming STM32 products.	This is required
openocd	OpenOCD is a community open source project that provides GDB remote debugging and flash programming support for a wide range of SoCs.	Already installed through Zephyr SDK if using Linux .

Note about JLink

Some of you may be tempted to use the STM reflasher to flash the JLink firmware onto the STLink. Please do not do that. These boards will eventually get passed down to 18-349, and it may break their workflow.

References

- [Nucleo F401RE](#)

Setting up your developer environment (every time you open a new terminal)

Every time you open a new terminal, you need to run the following:

Shell

```
# === Linux/Mac ===  
# Source the Python environment  
source .venv/bin/active  
  
# Source the Zephyr environment  
source zephyr/zephyr-env.sh  
  
# === Windows ===  
# Activate the Python environment  
.venv\Scripts\activate.bat  
  
# Source the Zephyr environment  
zephyr/zephyr-env.cmd
```

Flashing

Flashing your board is very simple. You just run the following command:

Swift

```
west flash
```

More information on the flash command can be found [here](#).

Debugging

Debugging on this board can be tedious. You will need **two** terminals. One for the debug server, and the other for the debugging session. Run the following commands:

Shell

```
# In terminal 1  
west debugserver  
  
# In terminal 2  
west debug
```

Build with debug options enabled

Shell

```
west build -b nucleo_f401re/stm32f401xe -- -DCONFIG_DEBUG_OPTIMIZATIONS=y  
-DCONFIG_DEBUG_THREAD_INFO=y <other cmake options>
```

Notes

You may run into a situation where the debugger seems like it is looping to the same breakpoint that you did not set when you start the debugging session. I don't know why this happens, however, there is a work around. See [GDB keeps pausing on `z\\_arm\\_reset\(\)`](#).

Troubleshooting

Permission denied in Linux

Problem: udev rules are not installed

Solution: Install the udev rules, reload them, and power cycle the board (unplug and plug back in)

Shell

```
wget -O 60-openocd.rules  
https://sf.net/p/openocd/code/ci/master/tree/contrib/60-openocd.rules?format=raw  
sudo cp 60-openocd.rules /etc/udev/rules.d  
sudo udevadm control --reload
```

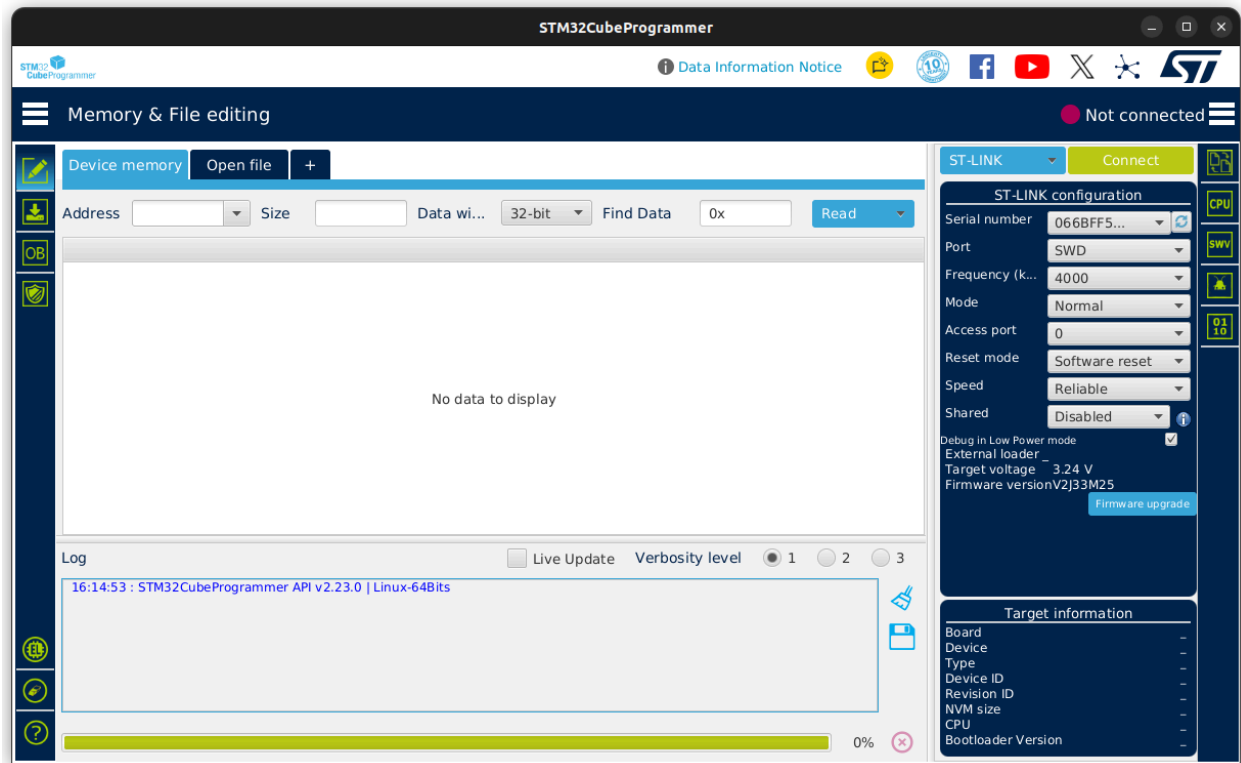
Weird error message if using openocd as a runner

Problem:

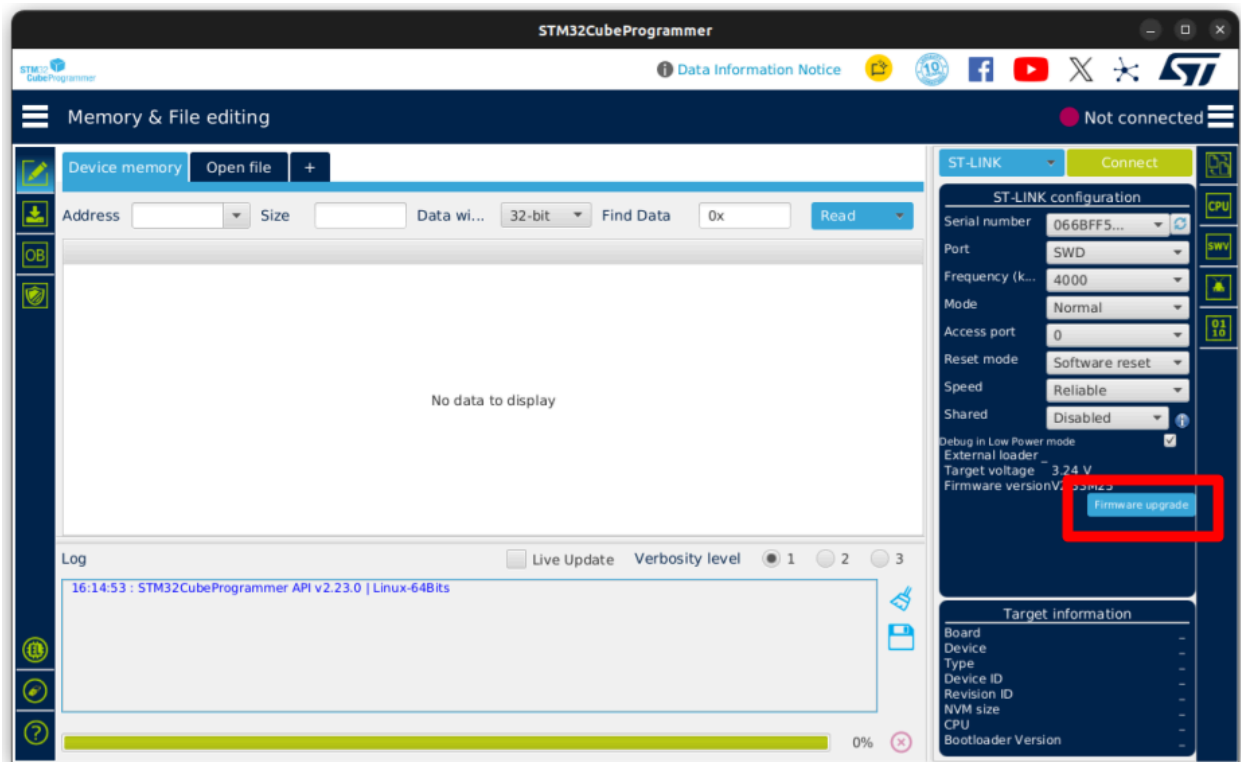
```
FATAL ERROR: command exited with status 1: /home/tschmitz/zephyr-sdk-1.0.1/hosttools/sysroots/x86_64-pokysdk-linux/usr/bin/openocd -s /home/tschmitz/wiselab/Adv-Embedded/zephyr-rtos/latest/zephyr/boards/st/nucleo_f401re/support -s /home/tschmitz/zephyr-sdk-1.0.1/hosttools/sysroots/x86_64-pokysdk-linux/usr/share/openocd/scripts -f /home/tschmitz/wiselab/Adv-Embedded/zephyr-rtos/latest/zephyr/boards/st/nucleo_f401re/support/openocd.cfg '-c in it' '-c targets' -c 'reset init' -c 'flash write_image erase /home/tschmitz/wiselab/Adv-Embedded/zephyr-rtos/latest/lecture/build/zephyr/zephyr.hex' -c 'reset run' -c shutdown
```

Solution: Upgrade the STLink firmware

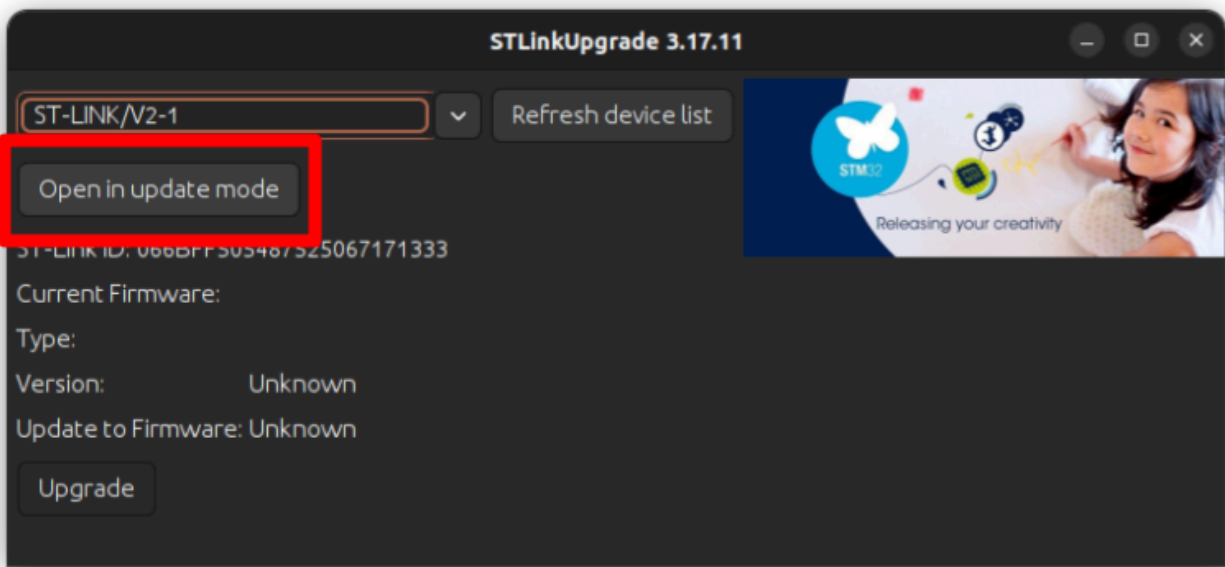
1. Open STM32CubeProgrammer



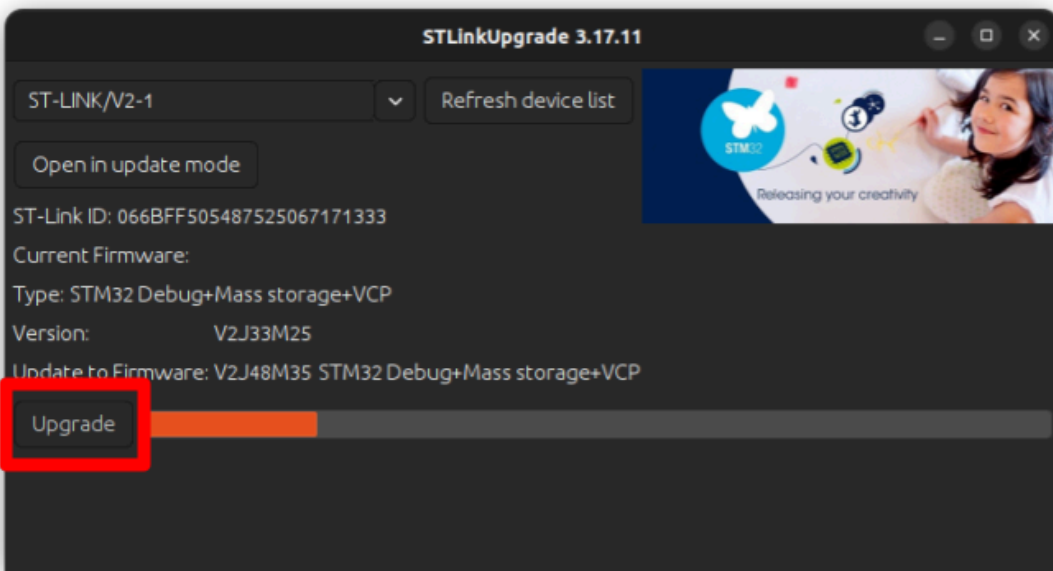
2. Click “Firmware Upgrade” button



3. Click the “Open in update mode” button



4. Click "Upgrade"



GDB keeps pausing on `z\_arm\_reset()`

Problem: GDB Keeps pausing on z_arm_reset()

```
147      ldr r0, =z_main_stack + CONFIG_MAIN_STACK_SIZE
Loading section rom_start, size 0x168 lma 0x8000000
Loading section text, size 0xd26c lma 0x8000180
Loading section .ARM.exidx, size 0x8 lma 0x800d3ec
Loading section initlevel, size 0xb0 lma 0x800d3f4
Loading section device_area, size 0x1a0 lma 0x800d4a4
Loading section sw_isr_table, size 0x250 lma 0x800d644
Loading section k_kernel_init_post_entry_area, size 0x4 lma 0x800d894
Loading section device_api_area, size 0x9c lma 0x800d898
Loading section zephyr_dbg_info, size 0x4c lma 0x800d934
Loading section shell_area, size 0x2c lma 0x800d980
Loading section shell_root_cmds_area, size 0x30 lma 0x800d9ac
Loading section shell_subcmds_area, size 0x104 lma 0x800d9dc
Loading section shell_dynamic_subcmds_area, size 0x8 lma 0x800dae0
Loading section rodata, size 0x240c lma 0x800dae8
Loading section datas, size 0xa8 lma 0x800fef4
Loading section device_states, size 0x1c lma 0x800ff9c
Loading section k_heap_area, size 0x14 lma 0x800ffb8
Loading section k_mutex_area, size 0x14 lma 0x800ffcc
Loading section .last_section, size 0x4 lma 0x800ffe0
--Type <RET> for more, q to quit, c to continue without paging--c
Start address 0x080046b0, load size 65484
Transfer rate: 28 KB/sec, 2976 bytes/write.
(gdb) b toggle_led
Breakpoint 1 at 0x8000c74: file /home/tschmitz/wislab/Adv-Embedded/zephyr-rtos/latest/lecture/src/main.c, line 27.
Note: automatically using hardware breakpoints for read-only addresses.
(gdb) c
Continuing.
jtag status contains invalid mode value - communication failure
[New Thread 536871648]
[Remote target exited]
[New Thread 536871240]
[New Thread 536872280]

Thread 2 "idle" stopped.
[Switching to Thread 536871648]
z_arm_reset () at /home/tschmitz/wislab/Adv-Embedded/zephyr-rtos/latest/zephyr/arch/arm/core/cortex_m/reset.S:147
147      ldr r0, =z_main_stack + CONFIG_MAIN_STACK_SIZE
(gdb) 
```

Solution: Set a breakpoint and force your program to hit that breakpoint

```
Loading section k_kernel_init_post_entry_area, size 0x4 lma 0x800d894
Loading section device_api_area, size 0x9c lma 0x800d898
Loading section zephyr_dbg_info, size 0x4c lma 0x800d934
Loading section shell_area, size 0x2c lma 0x800d980
Loading section shell_root_cmds_area, size 0x30 lma 0x800d9ac
Loading section shell_subcmds_area, size 0x104 lma 0x800d9dc
Loading section shell_dynamic_subcmds_area, size 0x8 lma 0x800dae0
Loading section rodata, size 0x240c lma 0x800dae8
Loading section datas, size 0xa8 lma 0x800fef4
Loading section device_states, size 0x1c lma 0x800ff9c
Loading section k_heap_area, size 0x14 lma 0x800ffb8
Loading section k_mutex_area, size 0x14 lma 0x800ffcc
Loading section .last_section, size 0x4 lma 0x800ffe0
--Type <RET> for more, q to quit, c to continue without paging--c
Start address 0x080046b0, load size 65484
Transfer rate: 28 KB/sec, 2976 bytes/write.
(gdb) b toggle_led
Breakpoint 1 at 0x8000c74: file /home/tschmitz/wislab/Adv-Embedded/zephyr-rtos/latest/lecture/src/main.c, line 27.
Note: automatically using hardware breakpoints for read-only addresses.
(gdb) c
Continuing.
jtag status contains invalid mode value - communication failure

Thread 3 "idle" stopped.
z_arm_reset () at /home/tschmitz/wislab/Adv-Embedded/zephyr-rtos/latest/zephyr/arch/arm/core/cortex_m/reset.S:147
147      ldr r0, =z_main_stack + CONFIG_MAIN_STACK_SIZE
(gdb) c
Continuing.
target not halted
target stm32f4x.cpu was not halted when resume was requested
Previous state query failed, trying to reconnect
jtag status contains invalid mode value - communication failure
Previous state query failed, trying to reconnect
[Switching to Thread 536872280]

Thread 2 "sysworkq" hit Breakpoint 1, toggle_led (ctx=ctx@entry=0x200000f0 <led_ctx>)
at /home/tschmitz/wislab/Adv-Embedded/zephyr-rtos/latest/lecture/src/main.c:27
27      static void toggle_led(struct led_ctx *ctx) {
(gdb) 
```

After it hits that first breakpoint, it should stop pausing on `z_arm_reset()`

```
tschmitz@tschmitz-MAC132C4M8: ~/wslab/Adv-Embedded/zephyr-rtos/latest/zephyr/arch/arm/core/cortex_m/reset.S$
Loading section shell_area, size 0x2c lma 0x800d980
Loading section shell_root_cmds_area, size 0x30 lma 0x800d9ac
Loading section shell_subcmds_area, size 0x104 lma 0x800d9dc
Loading section shell_dynamic_subcmds_area, size 0x8 lma 0x800dae0
Loading section rodata, size 0x240c lma 0x800dae8
Loading section datas, size 0xa8 lma 0x800fef4
Loading section device_states, size 0x1c lma 0x800ff9c
Loading section k_heap_area, size 0x14 lma 0x800ffb8
Loading section k_mutex_area, size 0x14 lma 0x800ffcc
Loading section .last_section, size 0x4 lma 0x800ffe0
--Type <RET> for more, q to quit, c to continue without paging--c
Start address 0x080046b0, load size 65484
Transfer rate: 28 KB/sec, 2976 bytes/write.
(gdb) b toggle_led
Breakpoint 1 at 0x0800c74: file /home/tschmitz/wslab/Adv-Embedded/zephyr-rtos/latest/lecture/src/main.c, line 27.
Note: automatically using hardware breakpoints for read-only addresses.
(gdb) c
Continuing.
jtag status contains invalid mode value - communication failure

Thread 3 "idle" stopped.
z_arm_reset () at /home/tschmitz/wslab/Adv-Embedded/zephyr-rtos/latest/zephyr/arch/arm/core/cortex_m/reset.S:147
147     ldr r0, =z_main_stack + CONFIG_MAIN_STACK_SIZE
(gdb) c
Continuing.
target not halted
target stm32f4x.cpu was not halted when resume was requested
Previous state query failed, trying to reconnect
jtag status contains invalid mode value - communication failure
Previous state query failed, trying to reconnect
[Switching to Thread 536872280]

Thread 2 "sysworkq" hit Breakpoint 1, toggle_led (ctx=ctx@entry=0x200000f0 <led_ctx>)
at /home/tschmitz/wslab/Adv-Embedded/zephyr-rtos/latest/lecture/src/main.c:27
27     static void toggle_led(struct led_ctx *ctx) {
(gdb) c
Continuing.
halted: PC: 0x0800c76
```

Notes

- This may take a few attempts. STLink and OpenOCD are not the best when it comes to debugging...
- Debugging the main thread works perfectly fine until it exits or sleeps forever. Then you are back in the same situation where things are constantly restarting.
- If things are still not working, you can try updating the STLink firmware. Steps found in the [previous section](#).
- If the debug server stops working, just stop it and restart it.